

KLAES VIRTUAL FILE SYSTEM (VFS) FRAMEWORK

Tuesday, September 22, 2026

Contents

Background	3
------------------	---

1. Backend Database Architecture (The Control Layer)	3
A. Core Entity Tables	3
B. The SPU/APU Engine Tables.....	3
C. The Virtual View Engine (The Magic Table)	4
2. Processing the SPU and APU (DPU) Workflows.....	4
3. Frontend UI Framework (The Virtual Display).....	5
A. The "Virtual File Explorer" Component	5
B. Handling the "Temporary File (T)" Scenario	5
4. Technical Implementation Rules (Best Practices)	6
Summary of the User Experience.....	6

Background

This is a classic Logical vs. Physical Data Architecture challenge. I want the physical EDMS to remain the "single source of truth" for document storage (preventing data duplication and storage bloat), while the application layer (KLAES) manages the complex legal and spatial lineage of the parcels.

To achieve this, we will design a Virtual File System (VFS) Framework. This framework uses a backend mapping layer to dynamically assemble "Virtual Folders" on the frontend based on the physical EDMS hierarchy, without ever creating physical duplicates.

Here is the comprehensive framework for the SPU and APU (DPU) Management System.

1. Backend Database Architecture (The Control Layer)

Instead of creating physical folders, we use relational tables to map Logical Folios (the legal/parcel entity) to Physical Files (the EDMS folder).

A. Core Entity Tables

1. `Physical\_Files` (Represents the actual EDMS Folders)

- `physical\_file\_id` (PK)
- `registry\_type` (Land / DCIV)
- `physical\_file\_no` (e.g., "1234", "1234(T)")
- `is\_temporary` (Boolean - for missing main files)
- `status` (Active, Archived, Superseded)

2. `Logical\_Folios` (Represents the current legal status of the parcel)

- `folio\_id` (PK)
- `current\_file\_no` (The active FileNo displayed to the user)
- `plot\_no`, `land\_use`, `coordinates`
- `status` (Active, Merged, Subdivided, Superseded)

B. The SPU/APU Engine Tables

3. `Parcel\_Transactions` (Tracks the SPU/APU events)

- `transaction\_id` (PK)
- `transaction\_type` (SPU or APU/DPU)
- `process\_sequence` (Integer: 1, 2, 3... used **only** for APU to track the order of operations)

- `process\_type` (Subdivision, Merger, Extension, CoP, Temp Commission)
- `approval\_date`, `status`

4. `Folio\_Lineage\_Mapping` (Tracks Parent-to-Child relationships)

- `mapping\_id` (PK)
- `transaction\_id` (FK)
- `parent\_folio\_id` (FK - The file(s) giving birth/merging)
- `child\_folio\_id` (FK - The new file born/created)
- Note: For a Merger, multiple rows will share the same `child\_folio\_id` but have different `parent\_folio\_id`s.

C. The Virtual View Engine (The Magic Table)

5. `Logical\_to\_Physical\_Page\_Mapping` (Tells the UI what to display)

- `mapping\_id` (PK)
- `folio\_id` (FK - The Logical Folio being viewed)
- `physical\_file\_id` (FK - The actual EDMS folder)
- `page\_index` (The image/page number)
- `page\_origin\_context` (e.g., "Original Main File", "Merger Source A", "New Survey Plan")
- `sort\_order` (Determines the display order in the virtual folder)

2. Processing the SPU and APU (DPU) Workflows

Handling SPU (Single Process)

When an SPU is approved (e.g., Subdivision of File A into B and C):

1. The system creates new `Logical\_Folios` for B and C.
2. It logs a `Parcel\_Transaction` (Type: SPU, Process: Subdivision).
3. It updates `Folio\_Lineage\_Mapping` (Parent: A -> Children: B, C).
4. Crucial Step: It copies the `Logical\_to\_Physical\_Page\_Mapping` from A, and assigns the relevant pages to B and C, appending any **new** pages (like the new survey plan) to the respective physical folders, and mapping them to B and C.

Handling APU / DPU (Multi-Process)

Because APU involves sequential steps, the system uses a Transaction Batch approach.

- Example (Scenario 1): CoP -> Merger -> Subdivision -> CoP.

- The system creates a single `transaction\_batch\_id`.
- It creates 4 sequential records in `Parcel\_Transactions` with `process\_sequence` 1, 2, 3, 4.
- The `Folio\_Lineage\_Mapping` is updated at **each step**.
 - Step 1: File 1 (COM) changes to File 1 (RES).
 - Step 2: File 1, File 2, File 3 merge into File 4. (Lineage maps 1, 2, 3 -> 4).
 - Step 3: File 4 subdivides into 39 files. (Lineage maps 4 -> 39 children).
 - Step 4: 3 of the 39 files change to COM.

3. Frontend UI Framework (The Virtual Display)

When a user clicks on a File (Folio) in the KLAES interface, the system should not query the physical EDMS directly. Instead, it queries the Virtual View Engine.

A. The "Virtual File Explorer" Component

When a Folio is opened, the UI renders a virtual folder structure based on the `Logical\_to\_Physical\_Page\_Mapping` table. To respect the physical hierarchy while showing the APU history, use a Grouped/Tabbed View:

- Tab 1: Current Active Documents (Pages mapped directly to the current Logical Folio).
- Tab 2: Lineage & History (The APU Order)
 - **Visual Timeline:** Displays the APU steps. `[CoP] -> [Merger] -> [Subdivision] -> [CoP]`.
 - **Clickable Nodes:** If the user clicks the "Merger" node in the timeline, the Virtual Folder temporarily filters to show **only** the pages that originated from the files involved in that specific merger step.
- Tab 3: Source Physical Folders
 - Displays the actual physical EDMS paths (e.g., `EDMS/Land Registry/1234/`) as read-only links. If the user needs to see the raw physical folder, they can click to open it in a new tab.

B. Handling the "Temporary File (T)" Scenario

- When a main file is missing, the system flags the `Logical\_Folio` as `Has\_Temp\_File = True`.
- The UI displays a banner: "Viewing Temporary File 1234(T). Main File 1234 is currently missing."
- The `Logical\_to\_Physical\_Page\_Mapping` points to the physical folder `1234(T)`.

- When the main file is found, an SPU (Merge) is processed. The system updates the mapping to point back to the main physical folder `1234`, and the physical folder `1234(T)` is marked as `Archived` in the EDMS.

4. Technical Implementation Rules (Best Practices)

1. No Physical Duplication Rule: The backend must strictly forbid the copying of image files (PDFs/JPEGs) from one physical folder to another during an SPU/APU. Only **database pointers** (the mapping table) should be updated. New documents (like a new survey plan for a subdivision) are the **only** new physical files created.
2. Session-Based Virtual Folders: As suggested, if a complex APU requires a temporary combined view, generate a Session ID on the frontend. Pass this Session ID to the backend, which temporarily caches the assembled virtual folder structure in Redis/Memory. Once the user closes the tab, the session expires. No physical folders are touched.
3. Audit Trail (Crucial for Land Registry): Because files are changing identities (File A becomes File B), the `Folio\_Lineage\_Mapping` table acts as an unbreakable audit trail. If a query is ever raised about "Where did the pages for File B come from?", the system traces it back: File B <- Subdivision <- File A <- Merger <- File X, Y, Z.
4. Performance Optimization: The `Logical\_to\_Physical\_Page\_Mapping` table will grow large. Ensure it is heavily indexed on `folio\_id` and `physical\_file\_id`. Consider implementing pagination on the frontend so that if a merged file contains 5,000 pages, the UI doesn't crash trying to load them all at once.

Summary of the User Experience

By implementing this framework, when a Land Registry staff member opens a complex APU file (e.g., a file that was merged, subdivided, and changed purpose), they will see a clean, unified virtual folder. They won't need to know the complex history to find the current documents, but if they **need** to investigate the history, the Lineage Timeline will allow them to step backward through the exact sequence of SPU/APU events, viewing the documents exactly as they existed at each stage of the process.